

Functional Safety for Thermal Management Systems

Final Documentation

Team C

Maria Anthonyos Hanna

Mohamad Abdo

Jonathan Bahry

Advanced Hardware Engineering

Prof.Ali Hayek

Hochschule Hamm Lippstadt

Department of Electronic Engineering

12.01.2026

Table of Contents

<i>I. Introduction.....</i>	<i>3</i>
<i>II. Concept Description</i>	<i>3</i>
<i>III. System Architecture and Hardware Platform</i>	<i>4</i>
<i>i. Hardware Components and Prototype Integration.....</i>	<i>4</i>
<i>ii. Clocking, Reset, and Observability.....</i>	<i>5</i>
<i>IV. Hardware Design and Development.....</i>	<i>5</i>
<i>V. Dual MicroBlaze System and Redundancy Strategy</i>	<i>6</i>
<i>VI. Software Architecture and Implementation</i>	<i>7</i>
<i>VII. Functional Safety and System Behavior</i>	<i>8</i>
<i>VIII. Hardware Schematic and Implementation Details.....</i>	<i>9</i>
<i>IX. Challenges and Limitations</i>	<i>10</i>
<i>X. Team Members Contribution.....</i>	<i>10</i>

I. Introduction

Battery systems are widely used in energy storage and electric mobility applications, where safe operation is a critical requirement. One of the main challenges in such systems is temperature control, as abnormal temperature behavior can lead to serious safety risks. For this reason, battery temperature monitoring plays an important role in modern Battery Management Systems (BMS).

This project focuses on the development of a battery temperature management system with an emphasis on functional safety. The system is implemented on a Xilinx FPGA platform using two MicroBlaze soft-core processors to introduce redundancy and improve fault detection. By comparing independent temperature measurements, the system can detect faults and move into a safe state when abnormal behavior is observed.

The following sections describe the system architecture, hardware and software implementation, and the safety mechanisms used to ensure reliable operation.

II. Concept Description

The main concept of the proposed system is to provide **reliable and safe battery temperature monitoring** using a redundant processing approach. The system is designed according to functional safety principles, where any detected inconsistency leads to conservative system behavior, and how would the 2 microblazes acts regards redundant data for human safety.

The temperature management system is based on an **asymmetric Dual Modular Redundancy (DMR)** architecture. Two independent processing units measure temperature in parallel using separate sensors. One processor acts as the **primary safety controller**, while the second processor operates as a **monitoring unit** that supplies redundant temperature data. Only the primary controller is allowed to make safety-critical decisions and control system outputs. Redundant temperature data is exchanged through a deterministic shared-memory mechanism, allowing continuous comparison between the two measurements. As long as both values remain within a predefined tolerance, the system continues normal operation. If a deviation beyond the safety threshold is detected, the system transitions into a **fail-safe shutdown state** to prevent unsafe operation.

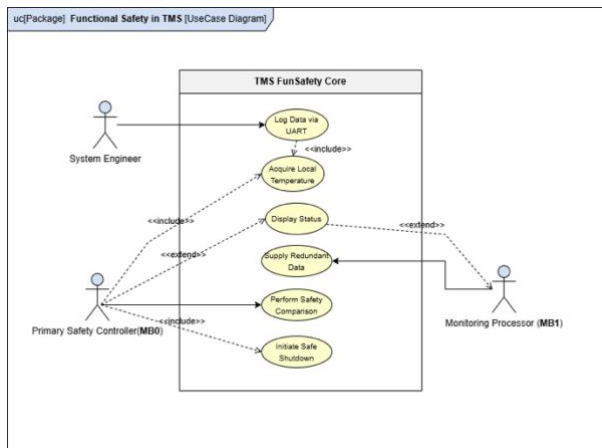
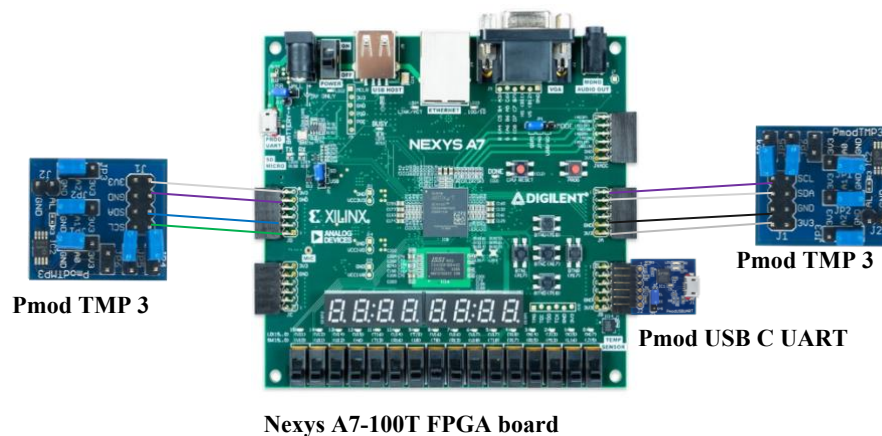


Fig.1 Use case diagram of the Functional Safety

Fig.1 presents a **use case diagram** that summarizes the main functional interactions of the system from a user perspective. The diagram highlights the key system functions, including temperature Data acquisition, redundancy verification, status reporting, and safe shutdown behavior.

III. System Architecture and Hardware Platform

i. Hardware Components and Prototype Integration



Nexys A7-100T FPGA board

Fig.2 Prototype hardware integration

The prototype implementation integrates the FPGA platform with external sensing and communication modules, as shown in Fig.2 . The **Nexys A7 100T FPGA board** serves as the central hardware component and hosts all processing and memory elements within the FPGA fabric. Temperature sensing is implemented using **two Pmod TMP3 digital temperature sensor modules**, each connected to a separate Pmod interface on the FPGA board. In the prototype setup, the **left Pmod TMP3 module is connected to the primary MicroBlaze controller (MB0)**, while the **right Pmod TMP3 module is connected to the monitoring processor (MB1)**. This configuration ensures **physical separation of the sensing paths**, which supports redundant temperature measurement.

The sensors communicate with the FPGA through an **I²C-compatible interface** implemented using general-purpose I/O pins. System monitoring and debugging are supported through

a **Pmod UART module**, providing serial communication between the FPGA system and an external host. In addition, the on board **seven segment display** is used to indicate system states such as normal operation, cooling conditions, and safe shutdown. This hardware configuration results in a compact and modular prototype suitable for validating the proposed temperature management and redundancy mechanisms.

ii. Clocking, Reset, and Observability

We designed the system to run on a **single global clock domain** to ensure deterministic timing and reliable operation of the redundant processors. The system clock is generated using the FPGA clock management resources and is distributed to both MicroBlaze processors and all connected peripherals. Using one common clock source avoids clock domain crossing problems and ensures consistent timing during temperature measurement and comparison.

An **active low reset** is used to provide a clean and predictable system startup. This reset method ensures that both processors and their peripherals start from a known state before normal operation begins, which is important for correct redundant behavior. To observe and test the system during operation, we included monitoring interfaces. A **Pmod UART module** is used to send system status and diagnostic messages to an external computer. In addition, the on-board **seven segment display** shows the current system state, such as normal operation, cooling mode, or safe shutdown. These features helped us to verify correct system behavior and validate the safety logic.

IV. Hardware Design and Development

We developed the hardware design in multiple stages to gradually reach the final redundant system while reducing integration risks. The development started with a simple architecture to validate basic functionality of one UART communication IP before extending to two UARTs systems. To make the design to supports redundancy and functional safety requirements.

In the first stage, we implemented a **single MicroBlaze system** on the FPGA platform. This initial design included one MicroBlaze processor, local **Block RAM (BRAM)** for program and data storage, a **Pmod TMP3 temperature sensor**, and a **Pmod UART module** for serial communication. This setup allowed us to verify correct processor operation, temperature acquisition, and communication with external devices.

For temperature sensor communication, we initially attempted to use the standard **AXI IIC peripheral** provided by the FPGA design tools. However, this solution resulted in timing and compatibility issues when interfacing with the Pmod TMP3 sensor. To address this problem, we implemented a **software-based, bit-banged I²C interface** using general-purpose I/O pins. This approach gave us full control over the I²C timing and ensured reliable communication with the temperature sensors.

After validating the single-processor system, we extended the hardware design to support a **dual MicroBlaze architecture**. Each MicroBlaze processor was provided with its own local memory and peripheral resources to minimize unintended interference and reduce fault propagation. A **shared BRAM** was added to enable controlled data exchange between the two processors, which is later used for redundant temperature comparison.

Throughout the hardware development process, all processing units and peripherals were kept within a **single clock domain**, and a consistent reset strategy was maintained. This step-by-step design approach helped us identify and resolve hardware integration issues early and provided a stable foundation for implementing the redundancy and functional safety mechanisms described in the following sections.

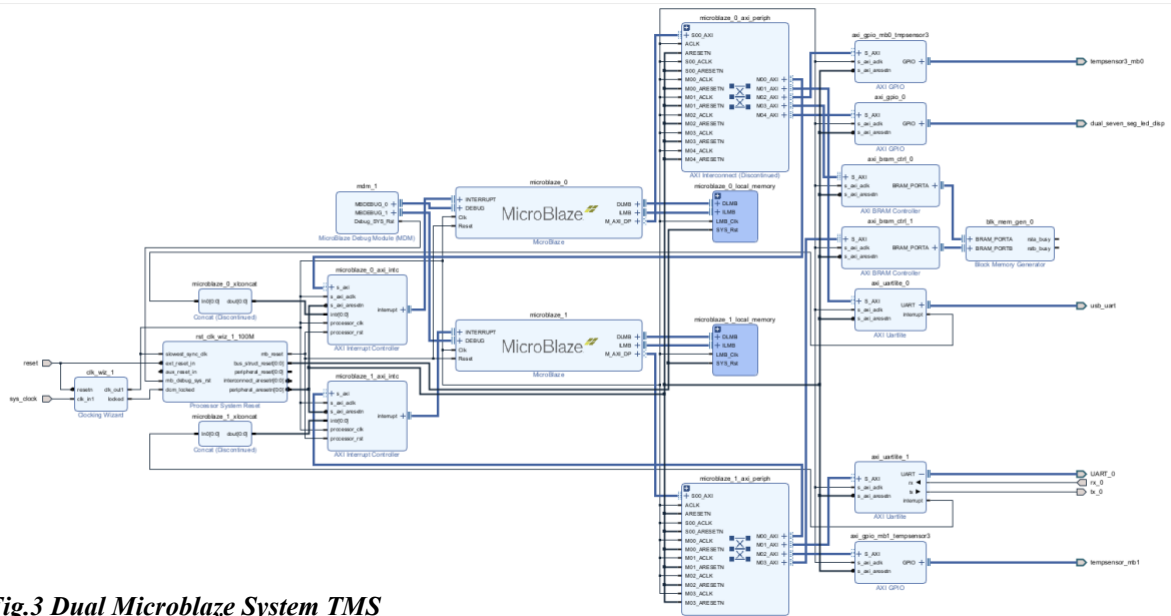


Fig.3 Dual Microblaze System TMS

V. Dual MicroBlaze System and Redundancy Strategy

To improve system reliability and support functional safety requirements, we extended the hardware design from a single processor to a **dual MicroBlaze architecture**. Both MicroBlaze

processors are implemented within the same FPGA device and operate in parallel while performing independent temperature measurements.

Each MicroBlaze processor is provided with its own **local Block RAM (BRAM)** for program and data storage, as well as independent peripheral interfaces. This separation reduces unintended interference between the processors and helps limit fault propagation. The two processors operate under a common clock and reset scheme to ensure synchronized and deterministic operation.

To enable communication between the processors, we implemented a **shared BRAM** that is accessible by both MicroBlaze systems. The shared memory is used to exchange temperature values and status information. The monitoring processor (MB1) periodically writes its measured temperature and a validity flag to the shared BRAM, while the primary controller (MB0) reads this data and performs a comparison with its locally measured temperature.

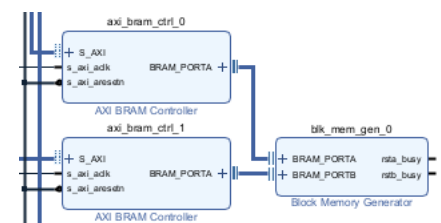


Fig.4 BRAM Comparator

The redundancy strategy follows an **asymmetric Dual Modular Redundancy (DMR)** approach. In this configuration, only the primary MicroBlaze processor (MB0) is allowed to make safety-critical decisions and control system outputs. The second processor (MB1) acts solely as a monitoring unit and does not directly influence system outputs. This design simplifies system control while still providing effective fault detection

If the difference between the temperature measurements exceeds a predefined tolerance (In our case 3) , MB0 interprets this condition as a fault, which may be caused by sensor failure, communication errors, or processing anomalies. In response, the system transitions to a **fail safe shutdown state** to prevent unsafe operation. This redundancy strategy ensures that any detected inconsistency leads to conservative system behavior rather than continued operation under uncertain conditions.

VI. Software Architecture and Implementation

Following the dual MicroBlaze architecture and redundancy strategy described in the previous section, we implemented a software structure that supports independent sensing and reliable

redundancy verification. Each MicroBlaze processor runs its own application, consistent with the defined master–monitor roles.

The software was developed using the **Xilinx Vitis environment**. The application on **MB0** is responsible for system supervision and safety-related decisions, while the application on **MB1** is limited to temperature acquisition and reporting.

Temperature sensors are accessed through a **GPIO based, bit banded I²C interface**, ensuring reliable communication with the Pmod TMP3 modules. Data exchange between the processors is performed through **shared BRAM**, which provides deterministic access for temperature comparison.

System status information is transmitted via the **Pmod UART module**, enabling observation and debugging during operation. This software structure directly supports the functional safety behavior described in the following section

VII. Functional Safety and System Behavior

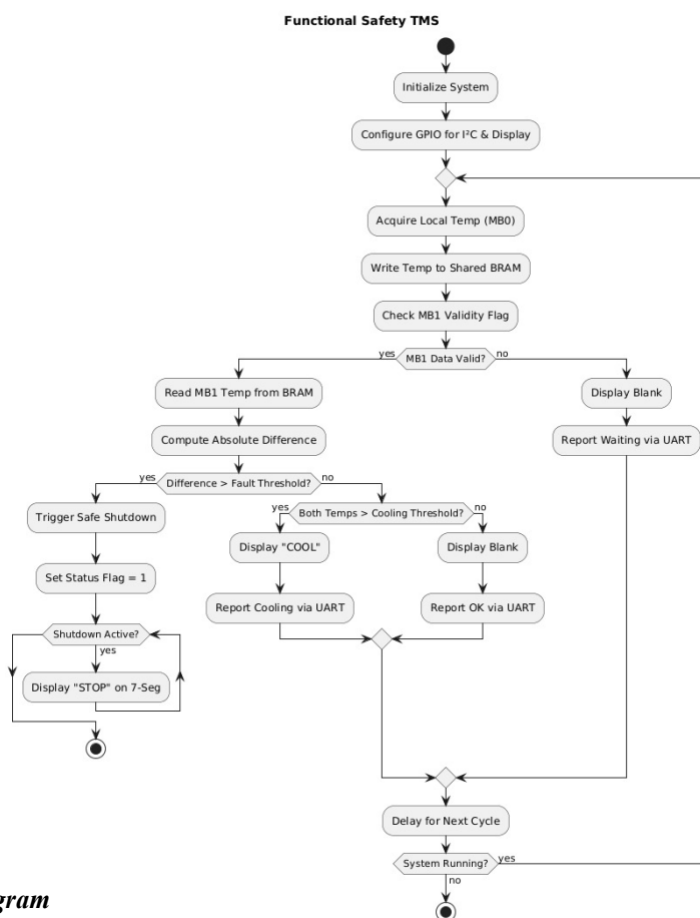


Fig 6. Activity Diagram

This Figure Fig.6 shows the safety control flow executed by the primary MicroBlaze processor (MB0). During each cycle, MB0 reads its local temperature, retrieves the redundant measurement from MB1 via shared BRAM, and compares the two values.

If the temperature difference exceeds the defined fault threshold, the system enters a **fail safe shutdown** state and displays a **STOP** indication. If no fault is detected but both temperatures exceed the cooling threshold, a **COOL** status is displayed. Otherwise, the system continues normal operation and reports an **OK** status.

This control loop runs continuously, ensuring reliable temperature monitoring and conservative system behavior in the presence of faults.

VIII. Hardware Schematic and Implementation Details

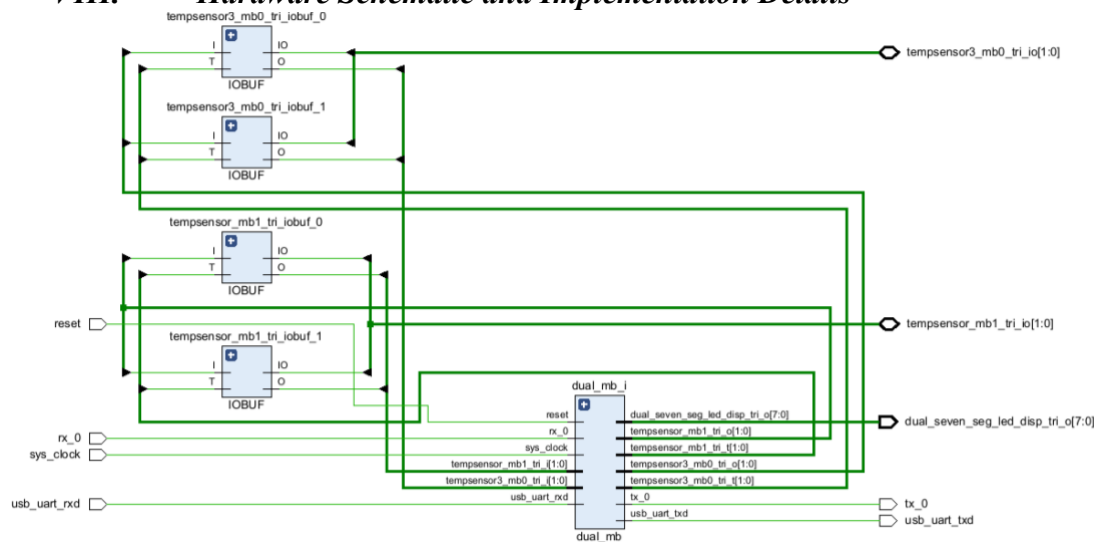


Fig 7. Activity Diagram

Fig 7. presents the hardware schematics generated from the FPGA design environment to support and verify the implemented system. The schematics provide an implementation-level view of the signal routing and external connections used in the final design.

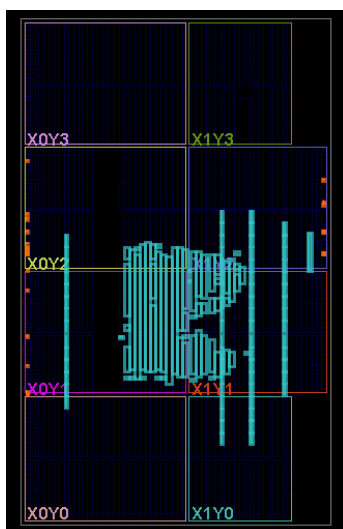


Fig 8. Netlist

This figure Fig 8. shows the post synthesis netlist and placement of our design on the FPGA.

IX. Challenges and Limitations

The main challenges during development were related to temperature sensor communication and dual-processor integration. The standard **AXI IIC interface** caused compatibility issues with the Pmod TMP3 sensors, which led to the use of a **GPIO-based, bit-banged I²C solution**.

Integrating the **dual MicroBlaze architecture** required careful configuration to ensure independent operation, and suitable **temperature thresholds** had to be selected to avoid false shutdowns.

Despite these challenges, the implemented solutions allowed the system to operate reliably and supported the functional safety objectives of the project.

X. Team Members Contribution

The project was carried out as a collaborative effort, with **all team members contributing to all stages of the project**, including system design, implementation, testing, and evaluation. In addition, each member took primary responsibility for coordinating specific parts of the work.

Mohamad Abdo primarily coordinated the **laboratory activities**, including experimental setup, hardware testing, and validation of the implemented system during lab sessions. His responsibilities included implementing and testing the **seven-segment display** and verifying correct system behavior in the lab environment.

Jonathan Bahry primarily coordinated the **main project development**, focusing on overall system design and integration. He worked closely with **Mohamad Aabdo** where Jonathan was mainly responsible for **hardware design in Vivado** and **software development and testing in Vitis**, including coding and debugging on the MicroBlaze processors.

Maria Anthonyos worked extensively on the **hardware connections and system integration**, ensuring correct wiring and interfacing of the FPGA board and sensors. In parallel, she coordinated the **documentation and reporting**, recording project steps, challenges, and results, and preparing the technical report and presentation.



Jonathan Bahry



Maria Anthonyos Hanna



Mohamad Abdo